

Chuleta compacta de Java

Consulta permitida durante los primeros microejercicios. No contiene soluciones a las pruebas.

Sintaxis esencial

Necesidad	Sintaxis / ejemplo	Resultado mental
Condición	<code>if (x > 0) { ... } else { ... }</code>	Una rama u otra
Bucle índice	<code>for (int i=0; i<list.size(); i++)</code>	<code>i: 0..size-1</code>
For-each	<code>for (String s : values)</code>	Recorre cada elemento
Retorno	<code>return result;</code>	Termina y devuelve
Ternario	<code>x >= 0 ? x : -x</code>	Valor según condición
Null	<code>Objects.requireNonNull(input)</code>	Falla pronto si es null
String igual	<code>Objects.equals(a, b)</code>	Compara contenido, tolera null

Colecciones

Objetivo	Estructura	Operaciones rápidas
Secuencia mutable	<code>List<T> x = new ArrayList<>();</code>	add, get, set, remove, size
Elementos únicos	<code>Set<T> x = new HashSet<>();</code>	add, contains, remove
Clave -> valor	<code>Map<K,V> x = new HashMap<>();</code>	put, get, getOrDefault, entrySet
Cola FIFO	<code>Queue<T> x = new ArrayDeque<>();</code>	offer, poll, peek
Pila LIFO	<code>Deque<T> x = new ArrayDeque<>();</code>	push, pop, peek
Prioridad	<code>PriorityQueue<T> x = new PriorityQueue<>();</code>	offer, poll

Métodos que debes reconocer

Tipo	Método	Ejemplo -> resultado / efecto
String	trim	" hola " -> "hola"
String	substring(1,3)	"java" -> "av"
String	split(",")	"a,b,c" -> [a,b,c]
String	contains	"hola java" -> true
String	isBlank	" " -> true
List	add/get/set	Añadir, obtener y reemplazar
Set	add	Devuelve false si ya existía
Map	getOrDefault	Valor o valor por defecto
Map	computeIfAbsent	Crear grupo si todavía no existe
Sort	list.sort(comparator)	Ordena y modifica la lista
Time	isBefore/isAfter	Compara horas/fechas
Optional	orElse/orElseThrow	Valor alternativo o error

Patrones de resolución

Problema	Herramienta mental	Coste típico
Contar frecuencias	Map<T,Integer>	O(n) tiempo / O(n) memoria
Detectar duplicados	Set<T>	O(n) promedio
Agrupar	Map<K,List<V>>	O(n) promedio
Ordenar	Comparator + sort	O(n log n)
Buscar en lista	Recorrido	O(n)
Intervalo [inicio,fin)	!t.isBefore(inicio) && t.isBefore(fin)	O(1)

Modelado y comprobación

```
record TimeRange(LocalTime start, LocalTime end) {}

enum Status { PENDING, PAID, CANCELLED }

interface Rule<T> {
    boolean accepts(T value);
}

List<T> copy = new ArrayList<>(input); // copiar antes de ordenar
copy.sort(comparator);

map.put(key, map.getOrDefault(key, 0) + 1);
map.computeIfAbsent(key, ignored -> new ArrayList<>()).add(value);
```

Checklist antes de tocar el teclado

Pregunta	Por qué importa
¿Puede ser null o estar vacío?	Define casos límite
¿Importa el orden de salida?	Decide HashMap/LinkedHashMap/ordenación
¿Puede haber duplicados?	Puede justificar Set o conteos
¿Puedo modificar la entrada?	Decide si debes copiar
¿Qué ocurre si no hay resultado?	null, Optional, vacío o error
¿Los límites están incluidos?	Clave para rangos y horarios

Uso progresivo

Vuelta 1: chuleta abierta. Vuelta 2: solo consultar tras intentarlo. Vuelta 3: cerrada durante 15 minutos. Simulación: cerrada completamente.